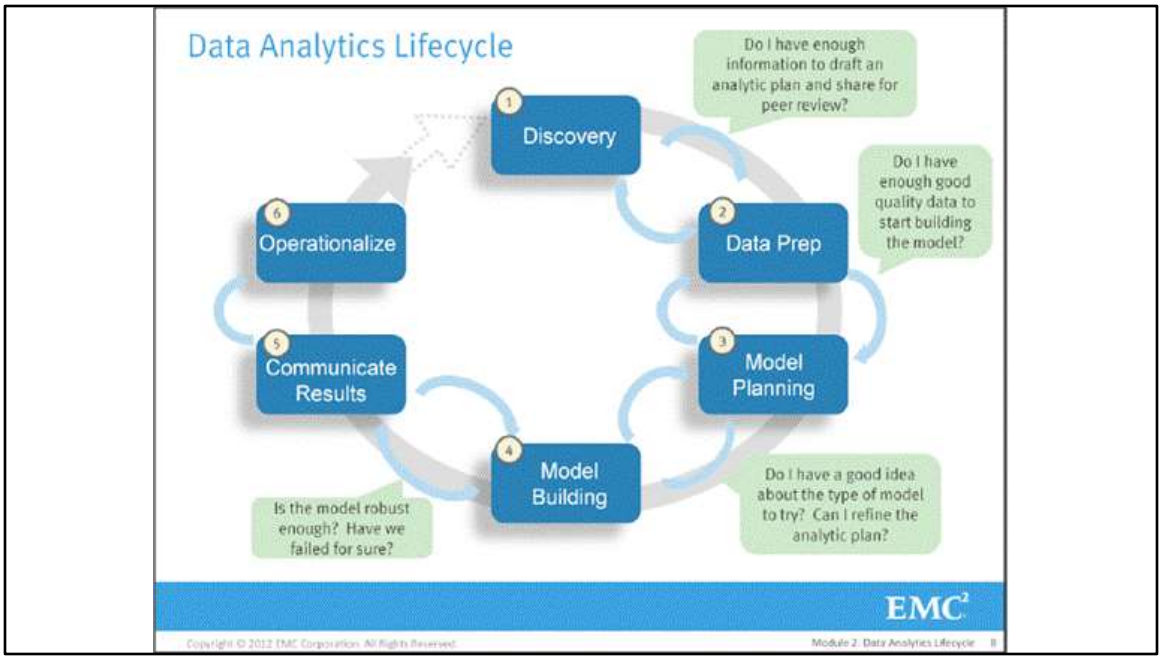
Data Preparation

How to Explore your Data and Prepare it for Model-Building

DSC 340

Feb. 14, 2025


Data Analytics Lifecycle
Do I have enough information to draft an analytic plan and share for peer review?
1 Discovery
2 Data Prep
Do I have enough good quality data to start building the model?
3 Model Planning
Do I have a good idea about the type of model to try? Can I refine the analytic plan?
4 Model Building
Is the model robust enough? Have we failed for sure?
5 Communicate Results
6 Operationalize
EMC²
Copyright © 2012 EMC Corporation. All Rights Reserved.
Module 2: Data Analytics Lifecycle 8

Data Prep

- GIGO
- Most time-consuming step (up to 50% of team's time)

Data Prep Overview [EMC 2.3]

1. Set up **sandbox**
   - Get data out of production environment!
   - Get data into a **stable form** (OLAP vs. OLTP)
   - Web **scraping** [that is, "scrape the web"; not *scrapping*!]
   - Classroom project: Download or `read_csv` from Kaggle, GitHub, etc.

Data Prep Overview [EMC 2.3]

2. Get to **know your data**
   - **Catalog** data sources available to the team
   - Make sure you have data needed to solve problem!
   - Are there **gaps**? Where can we find the data we need? Outside organization?
   - What are these **features**? How collected? When? Units? What does it mean?





https://www.stack.com/a/football-conditioning-drills-2

https://www.themuseatdreyfoos.com/top-stories/2015/10/14/preseason-soccer-conditioning/

Data Prep Overview (cont.) [EMC 2.3]

3. **Data conditioning:** Get your data in shape!**
   - **Flatten** ("denormalize"): **merge** SQL → table with rows and columns
   - **Structure**: by extracting features (e.g., clickbait detection)
   - **Clean:** Remove or correct data that is invalid, repeated, irrelevant …
   - **Transform:** Ex.: multiple time formats → Pandas timestamp
   - **Scale (or Normalize):** Ex.: Scale to [0, 1] so features have same scale*

=> "Feature Engineering"

*Not before train-test split (more on that later)

**Role of data engineer and IT extra important in this phase

Flatten: SQL tables Students, Rooms, Buildings, RoomAssignments, RoommateSurveys. (Suppose predicting roommate happiness.)

RGB Bitmap: 800x600x3: color channels [0,255] = 24-bit color. Flatten into a single row (vector) and maybe reduce resolution before applying Deep Learning.

Feature engineering: stomach temperature readings out of range from drinking cold water. In general: fixed basis functions, e.g., use Haversine and Census data to find distance to nearest MSA city center.

Problems: Too much to store in main memory all at once (use DB or sampling algorithm).

Conditioning: Questions to Ask

- **What are the sources?** (e.g., columns, class label)
- **How clean is the data?**
- **How consistent are contents and files?** (spreadsheet for ea. month)
- **Is there missing data?** (columns with no data; missing days: 2/29!)
- **Is everything the same type?** (numbers and text in Excel column)
- **Does this information make sense?** (0s, negative values)
- **Are there indicators of systematic error?** (broken sensor feeds)

Data Prep Overview (cont.) [EMC 2.3]

4. Survey and Visualize

- Get "high-level" view of data ("view from 40,000 feet")
- "Overview first, zoom and filter, then details-on-demand"
- Examples: Seaborn all-pairs XY-plot, Pandas df.describe(), sample photos

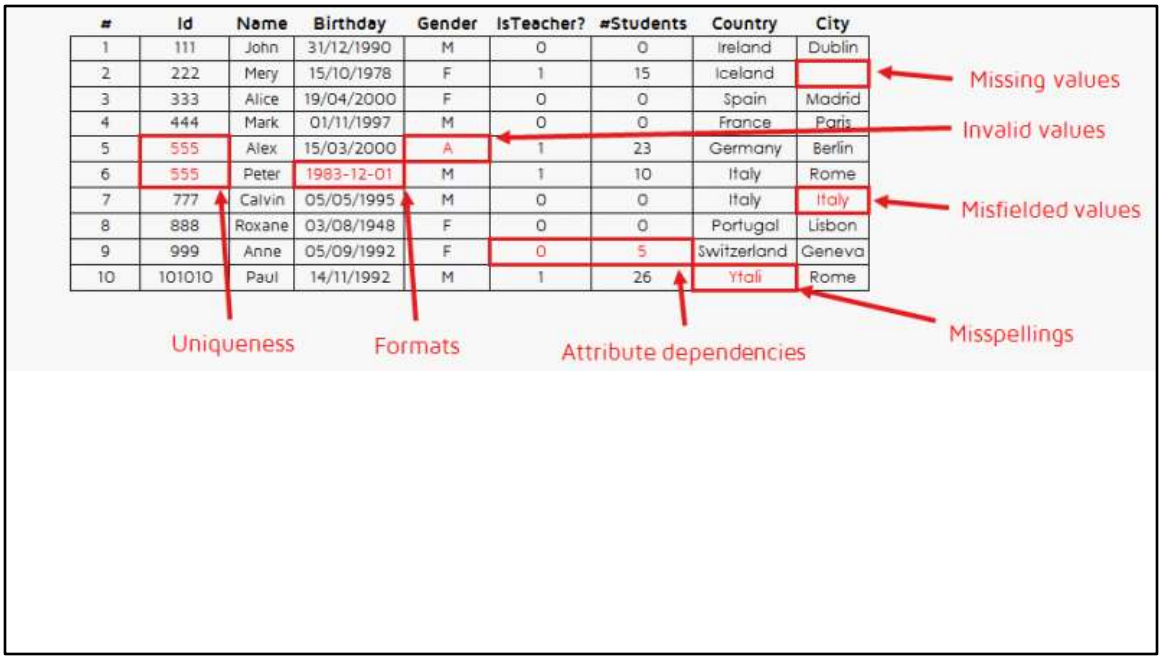
Checklist:

- Are calculations consistent within columns? Across tables?
- Is the data distribution consistent? If not, how to address?
- Does the data represent the population of interest? (kids, voters, etc.)
- Granularity consistent? (daily vs. every second)

Tools

- Pandas
- Seaborn and Matplotlib
- Descriptive stats: df.describe()
- .max() .min() .idxmax() .unique() or set()
- .groupby()
- df.sample(), np.random.choice()
- Scatter (XY) plot
- Dirty data (next slide)

df.unique() returns unique values (not a count)



#	Id	Name	Birthday	Gender	IsTeacher?	#Students	Country	City
1	111	John	31/12/1990	M	0	0	Ireland	Dublin
2	222	Mery	15/10/1978	F	1	15	Iceland	
3	333	Alice	19/04/2000	F	0	0	Spain	Madrid
4	444	Mark	01/11/1997	M	0	0	France	Paris
5	555	Alex	15/03/2000	A	1	23	Germany	Berlin
6	555	Peter	1983-12-01	M	1	10	Italy	Rome
7	777	Calvin	05/05/1995	M	0	0	Italy	Italy
8	888	Roxane	03/08/1948	F	0	0	Portugal	Lisbon
9	999	Anne	05/09/1992	F	0	5	Switzerland	Geneva
10	101010	Paul	14/11/1992	M	1	26	Ytali	Rome

Flattening Data

- **Relational databases** (RDBs) are **"normalized"**
  - Kevin's spreadsheet → many small, interrelated tables
  - Avoid duplicate data, reduce missing values
  - Promotes data integrity
- But for ML, we want a **matrix** (structure), i.e., **"flat"**
- If still in RDB: write SQL queries (**JOIN**, view, etc.)
- If in multiple DFs in Pandas: **.merge() .groupby()**
- Sometimes called "denormalization" but "normalize" overloaded here
  - In ML "normalization" often means $N(0,1)$

Scaling and Normalizing Data

- Suppose some data in ***km***, others ***m***
- **Scale differences can throw off many ML algorithms**
- **In other cases (e.g., logistic regression) can reduce efficiency**

"I tried SVM, but it took hours!" Me: "Did you scale your data?"

- **Min-Max Scaler:** $(\mathbf{X} - \mathbf{x}_{\min})/(\mathbf{x}_{\max} - \mathbf{x}_{\min})$ [0,1]
- **Standard Scaler:** $(\mathbf{X} - \bar{\mathbf{x}})/\mathbf{s}_{\mathbf{x}}$ $Z \sim N(0,1)$
- **Robust Scaler:** $(\mathbf{X} - \mathbf{x}_{\text{median}})/\text{IQR}$ interquartile/interdecile range
- Scikit-learn: MinMaxScaler, StandardScaler, RobustScaler

(Yes, sometimes it makes sense to use different scalers for different columns.)

Examples

```
>>> from sklearn.preprocessing import MinMaxScaler
>>> data = [[-1, 2], [-0.5, 6], [0, 10], [1, 18]]
>>> scaler = MinMaxScaler()
>>> print(scaler.fit(data))
MinMaxScaler()
>>> print(scaler.data_max_)
[ 1. 18.]
>>> print(scaler.transform(data))
[[0.   0.  ]
 [0.25 0.25]
 [0.5  0.5 ]
 [1.   1.  ]]
>>> print(scaler.transform([[2, 2]]))
[[1.5 0. ]]
```

https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html

Preprocessing Type	Scikit-learn Function	Range	Mean	Distribution Characteristics	When Use	Definition	Notes
Scale	MinMaxScaler	0 to 1 default, can override	varies	Bounded	Use first unless have theoretical reason to need stronger medicine.	Add or subtract a constant. Then multiply or divide by another constant. MinMaxScaler subtracts the minimum value in the column and then divides by the difference between the original maximum and original minimum.	Preserves the shape of the original distribution. Doesn't reduce the importance of outliers. Least disruptive to the information in the original data. Default range for MinMaxScaler is 0 to 1.
Standardize	RobustScaler	varies	varies	Unbounded	Use if have outliers and don't want them to have much influence.	RobustScaler standardizes a feature by removing the median and dividing each feature by the interquartile range.	Outliers have less influence than with MinMaxScaler. Range is larger than MinMaxScaler or StandardScaler.
Standardize	StandardScaler	varies	0	Unbounded, Unit variance	When need to transform a feature so it is close to normally distributed.	StandardScaler standardizes a feature by removing the mean and dividing each value by the standard deviation.	Results in a distribution with a standard deviation equal to 1 (and variance equal to 1). If you have outliers in your feature (column), normalizing your data will scale most of the data to a small interval.
Normalize	Normalizer	varies	0	Unit norm	Rarely.	An observation (row) is normalized by applying l2 (Euclidian) normalization. If each element were squared and summed, the total would equal 1. Could also specify l1 (Manhattan) normalization.	Normalizes each sample observation (row), not the feature (column)!

https://towardsdatascience.com/scale-standardize-or-normalize-with-scikit-learn-6ccc7d176a02

Encoding Nominal Data

- Two values (owner/renter): encode binary data as 0-1
- Multiple values (location A, B, C): create $(K-1)$ new columns
  - **One-hot encoding** (**dummy** variables) One-cold encoding also exists!
  - Create K (or $K-1$) new cols of $\{0,1\}$ with just one 1 across
  - `pd.get_dummies()` (sklearn) `OneHotEncoder`
- Ordinal data: encode as integer
  - Likert scale in survey (strongly agree, agree, etc.) can encode as 1-5
  - Feature "Year": First year, Sophomore, Junior, Senior (?)
- Sometimes numeric: below threshold is 0, above is 1 (class labels)

WE STOPPED HERE

Dealing with Missing Data

- Ignore it (filter out rows) `df.drop_duplicates()` `df.duplicated()`
- Drop variables (> 60%) `pd.drop(columns=["x1", "x2"], axis=1)`
- Mean (median, mode) **imputation** `df.fillna()` `SimpleImputer`
- Substitution (with 0, default value)
- Hot deck imputation (select random variable from column)
- Cold deck imputation (replace with hand-picked example)
- Regression imputation (predictive model)
- Interpolation and extrapolation `backfill bfill ffill`

CAUTION: Are you changing the distribution?

- **Why are the data missing?** (hypothesis)
  - Sensor glitched due to power outage
  - Weather station destroyed in storm
  - Athlete's first appearance (no prior data)
  - Older respondents skipped question #13
  - Some respondents didn't check either M or F box
  - Suspect's geolocation data missing at time of murder
- Classes of missing data: MCAR, MAR, MNAR
- **Justify your decision in notebook: reasoned explanation**
  - Even better, test your hypotheses (e.g., MCAR sets A and B, then *t*-test)
  - Professional settings: consult with statistician

Missing completely at random (MCAR): other data statistically the same: imputation methods are OK but count as additional estimated parameter
Missing (conditionally) at random (MAR): can predict which instances will have this missing
Missing Not at Random (MNAR): missing values cannot be inferred from available data: survey design issue
How many instances are affected? Percentage of entries?

Data Preparation **is** Statistical Modeling

- Modeling is not just calling .fit()!
- **Data preparation is statistical modeling**
- Ask why (hypothesize)
- Ideally, compare instances missing with non-missing: pattern?
- Consider the impact of dropping, imputing on your model
- Imputation can (will) affect variability in your data
- Caveat: Time on project usually limited: assess importance
- **In notebook: discuss reasoning, include exploration, justify actions**